RAG Assignment: Baseline Retrieval vs Re-Ranking

Nikhil Patidar
Roll No. 2025201081

October 19, 2026

Code: `https://github.com/Nikhil3028/RAG_Assignment`

Setup

For the document I used the FIDE Laws of Chess (2023). It is 41 pages long and is written like a legal text, with numbered clauses, many cross references, exceptions and separate appendices for rapid, blitz and other formats. A lot of rules use the same words (clock, move, draw, arbiter), so it is a good test for retrieval.

Item	What I used
Chunking	Split on clause numbers, around 700 characters per chunk, 124 chunks
Bi-encoder	sentence-transformers/all-MiniLM-L6-v2, FAISS inner product index (vectors are normalised, so this is cosine similarity)
Cross-encoder	cross-encoder/ms-marco-MiniLM-L-6-v2
LLM	Gemini (gemini-3.8-flash), temperature 0
Retrieval	Top 10 from the bi-encoder, Top 3 go to the LLM

In Pipeline 1 the bi-encoder ranks 1 to 3 go straight into the prompt (Answer 1). In Pipeline 2 I take the same 10 candidates, score all 10 (query, chunk) pairs with the cross-encoder, sort by that score and send the new top 3 (Answer 2). Both use exactly the same prompt, so the only thing that changes between the two answers is the context.

1 Architecture Comparison

A bi-encoder encodes the query and each chunk on their own. All chunk vectors are made once, before any question comes in, and stored in the index. When a question comes, only the question is encoded and I just look for the nearest vectors with cosine similarity. This is very fast, but the query and the chunk never meet inside the model. Each chunk becomes one fixed vector, so small words that change the meaning, like “not”, “except” or “automatic”, barely move the score. A chunk that shares a lot of words with the question can score high even if it does not answer it.

A cross-encoder gets the query and the chunk together as one input. Self-attention works across the tokens of both, so each word in the question can be compared with each word in the chunk, and the model gives one relevance score for the pair. It is much better at telling if a chunk really answers the question. The downside is that nothing can be prepared in advance: the model has to run once for every (query, chunk) pair at query time.

2 Qualitative and Rank Shift Analysis

2.1 Rank shifts

I wrote the failure queries for this document and checked them against the real retrieval output. For each one the gold chunk is the chunk that holds the clause with the answer. Chunks are named by the clauses they contain.

ID	Type	Gold chunk (clauses)	Baseline rank	Re-ranked rank
KT1	Keyword trap	6.10.2 to 6.11.3	#8	#1
KT2	Keyword trap	7.5.2 to 7.5.4	#8	#1
EX1	Exception	6.9 to 6.10.1	#7	#2
EX2	Exception	9.6.2	#4	#3
NG1	Negation	12.4 to 12.6	#4	#3
BU1	Buried chunk	7.3 to 7.5	#8	#3
CTRL	Control	7.1 to 7.2.2	#3	#7

In six of the seven queries the gold chunk was outside the baseline top 3, so the LLM never got to see it. After re-ranking it was inside the top 3 every time. The queries and why the bi-encoder got them wrong:

- KT1, "Can a player stop the clock to go to the toilet?" The word "clock" pulled in all the clock handling clauses (6.2.x). The answer (6.11.2) uses different words: a player may "pause the chessclock only to seek the arbiter's assistance".
- KT2, "Can a player use two hands to castle?" "Castle" matched the castling rules in Articles 3 and 4. The rule about two hands is in Article 7 with the illegal moves (7.5.4).
- EX1, "If a player runs out of time, do they always lose the game?" The general time rules came first. The exception, that the game is a draw if the opponent cannot checkmate, is in 6.9.
- EX2, "Is the draw automatic after 50 moves without capture?" The 50 move claim rule (9.3) was rank 1. The automatic draw only happens at 75 moves (9.6.2).
- NG1, "What is the arbiter not allowed to tell a player during the game?" "Not allowed" matched clauses about what players may or may not do (11.2.x). Clause 12.6 is about what the arbiter "must not" do.
- BU1, "Can a player be penalised for knocking pieces over?" In the document "knocking over" is only used for the clock (6.2.4). The rule for pieces says "displaces" (7.4.3).
- CTRL, "Can the time on the clock be corrected after an illegal move?" I kept this one on purpose because here the cross-encoder made things worse.

To see if this holds beyond the queries I picked, I also scored both rankings on 36 labelled questions (normal ones plus the hard ones):

Metric	Baseline	Re-rank
Hit@3 (gold chunk reaches the LLM)	0.611	0.889
MRR@10	0.567	0.718
Recall@10	0.972	0.972

Recall@10 does not change because both pipelines start from the same 10 chunks. All the gain comes from moving the right chunk into the top 3.

2.2 Answer 1 vs Answer 2

- KT2, EX1, NG1 and BU1: Answer 1 was just "The provided context does not contain this information." Answer 2 gave the right rule and cited the right clause (7.5.4, 6.9, 12.6 and 7.4.3). Here the baseline left out the essential clause because it was below rank 3.
- EX2: Answer 1 was not wrong, it said the draw has to be claimed. But it was incomplete and could easily be read as "there is no automatic draw at all". Only Answer 2 added that the draw becomes automatic at 75 moves (9.6.2).
- KT1: Answer 1 had nothing. Answer 2 had the right clause and said the clock can only be paused to ask the arbiter for help, but it still started with "does not contain this information" before giving the rule. So the context was fixed, but the LLM was too careful with its wording.

- CTRL: re-ranking hurt here. Clause 7.1 says the arbiter uses best judgement to set the clock times, and it was at rank 3 in the baseline, so Answer 1 was correct. The cross-encoder pushed it down to rank 7 and Answer 2 said the context does not have the answer.

I did not see a real hallucination in this run. My prompt tells the model to use only the context and to say so when the answer is missing, so when the right chunk was not there the model mostly refused instead of making up a rule. The failure was an omitted clause or a missing answer rather than a misstated rule. Overall re-ranking fixed six of the seven cases and broke one, so it helps a lot but it is not always right.

3 Latency Profiling

All timings are from a Google Colab CPU runtime. The two retrieval stages were run 5 times over all 36 questions (180 runs each). The LLM times come from the 7 failure queries.

Stage	Mean (ms)	Median (ms)	p95 (ms)
Stage 1: embed query and FAISS top 10	18.9	17.5	26.8
Stage 2: cross-encoder on 10 pairs	825.4	770.6	1169.9
Retrieval total, Pipeline 1	18.9	17.5	26.8
Retrieval total, Pipeline 2	844.3	787.2	1196.1
LLM generation, Pipeline 1	4281.9	2812.2	10321.6
LLM generation, Pipeline 2	3184.1	2119.0	7574.2
End to end, Pipeline 1	4299.4	2829.7	10339.1
End to end, Pipeline 2	3971.3	2906.3	8361.4

Re-ranking makes retrieval roughly 45 times slower (median 17.5 ms vs 787 ms). But the Gemini call alone takes 2 to 3 seconds, so for the full answer both pipelines take about the same time. The LLM time also jumps around a lot between calls because of the network and API load, which is why Pipeline 2 even looks a bit faster end to end. That is just noise. On a GPU the cross-encoder step would be much quicker than on this CPU.

4 Scaling Question

To see how the cross-encoder grows, I timed it on more and more chunks for the same query:

Chunks scored	Time
10	0.69 s
25	1.84 s
50	4.19 s
124 (whole document)	13.75 s

The time goes up in a straight line, about 0.11 s per chunk on this CPU. Even this single 41 page document takes almost 14 seconds per question. At 100,000 chunks it would be around 3 hours for one question, and at 10 million chunks around 13 days.

So we do not run the cross-encoder on everything from the start because none of its work can be done ahead of time. Every new question needs a full model pass for every chunk. The bi-encoder does its heavy work (encoding all the chunks) once, offline, and at query time it only encodes the question and does a vector search, which stays in milliseconds even for millions of chunks with an approximate index. That is why the two stage setup makes sense: the cheap bi-encoder cuts the collection down to a few candidates (10 here), and the expensive but accurate cross-encoder only looks at those.